Benchmarking LLM capabilities for ontology development and case extraction from predictive maintenance literature: An OntoCast study informed by the OPMAD ontology

***Abstract*—This work presents an experimental comparison of the capabilities of humans (OPMAD authors) and large language models (LLMs) for the tasks of ontology development and case extraction from predictive maintenance literature. LLMs were inferenced via the OntoCast Python library using a tailored extraction prompt constrained by a human-made ontology (OPMAD), and tasked to extract predictive maintenance cases and their associated facts from a corpus of 102 papers that had been previously extracted by the same human authors of the OPMAD ontology. The LLM- and human-extracted facts were compared using quality metrics. Then, the same 102-paper corpus was used to evolve a predictive maintenance case ontology from zero using OntoCast, whose quality was compared against OPMAD.**

I. Results

The seven pilot experiments reported here use OntoCast 0.8.2 in facts-only mode with the fixed OPMAD vocabulary and ontology evolution disabled. They cover five papers from the 102-paper corpus. The full-corpus and ontology-development comparisons described in the abstract remain outside these pilot results. Sol and Terra below denote the recorded model names `gpt-6.1-sol` and `gpt-6.1-terra`. Each quoted prompt reproduces the user extraction instruction. OntoCast also supplies its task and ontology context. Table I compares case counts and pipeline times.

For the comparisons below, S001 denotes the aircraft-engine prognostics study by Lu *et al.* [1]. S003 denotes the fuzzy early warning study by Vafaei *et al.* [2]. S004 denotes the fuel-cell degradation study by Wu *et al.* [3]. S007 denotes the acoustic diagnosis study of circuit breakers and tap changers by Hussain *et al.* [4]. S017 denotes the bearing remaining useful life study by Ren *et al.* [5]. The OPMAD authors extracted 6, 1, 3, 4, and 3 cases from these papers, respectively: 17 in total. These counts form the fixed human reference.

A *chunk* is a contiguous portion of document text processed as a separate content unit for fact extraction, rather than a maintenance case. OntoCast converts the PDF with Docling, detects section spans from the exported headings, and assigns academic section labels using heading rules and conservative content heuristics. Unrecognized sections can remain unlabeled. Long section spans are split semantically. Short neighboring pieces are joined within the same section label by default. The initial size settings target 3,000–12,000 characters, but section boundaries can leave shorter chunks. Section filtering precedes final sizing, and bibliography chunks are skipped before extraction.

Fact merging combines records that refer to the same thing across the whole paper. Each chunk initially gives its extracted entities separate identifiers. OntoCast looks for possible matches using embeddings, which are numerical representations of entity descriptions. The usual similarity threshold is 0.7. The records must have compatible types and roles, such as both being instances rather than classes. A merge also needs matching or similarly worded names, called lexical aliases, or a shared identifying value such as an equipment code. Accepted records receive one shared identifier, and their facts are updated to use it.

Additional checks keep distinct entities apart. OntoCast rejects merges between entities directly linked to each other and between entities listed separately under the same relationship. Conflicting values also block a merge. For a relationship that allows only one target, records pointing to different targets are kept apart. Different initials in otherwise matching names can also block a merge. High embedding similarity alone is therefore insufficient. Fact merging can compare records from different sections. Only the earlier step of joining text chunks is limited to neighboring chunks with the same section label. Some merge checks cannot be disabled.

A. E1: Minimal Prompt

Rationale: The first experiment tested whether a minimal extraction instruction was sufficient for Sol to recover the cases in S001 using the supplied ontology. The prompt was: “Extract every predictive maintenance case reported in this paper and all case-related facts expressible using the supplied ontology.”

Results: Extraction included cases from cited studies rather than only the paper’s own cases. A *cached replay* reruns the pipeline using saved LLM responses to identical requests instead of generating new responses. This replay yielded 21 case nodes against six human cases. This motivated an explicit restriction on which paper could supply cases. The original experiment’s runtime is unavailable. The replay time excludes LLM response generation and therefore does not measure the speed of a new model run.

B. E2: Restricting Extraction to the Paper’s Own Cases

Rationale: The second experiment repeated S001 with Sol, added an explicit exclusion of cited papers, and raised the request timeout to 600 seconds. The prompt was: “Extract every predictive maintenance case reported in this paper, not in

other papers cited within it, and all case-related facts expressible using the supplied ontology."

Results: The output contained four case nodes against six human cases, so its case count fell below the reference. The recorded pipeline time was about 8.0 minutes, with 13 cached responses and three fresh model calls. This mixed-cache run is not a fresh-inference speed benchmark.

C. E3: Extending the Pilot to Four More Papers

Rationale: The third experiment gathered evidence beyond S001 by applying Sol to four new papers: S003, S004, S007, and S017. It retained the source-restricted prompt: "Extract every predictive maintenance case reported in this paper, not in other papers cited within it, and all case-related facts expressible using the supplied ontology."

Results: The four papers yielded 64 case nodes against 11 human cases, with higher counts for every paper. The extractions contained duplicate cases, and PDF conversion problems were observed. The definition of a case remained ambiguous. Recorded pipeline time totaled about 74.6 minutes. These observations motivated both the model comparison and the later explicit case definition.

D. E4: Comparing Terra with Sol

Rationale: After the roughly 75-minute four-paper Sol run, the fourth experiment tested Terra on all five papers to assess whether extraction could be faster while retaining useful output. The extraction prompt was unchanged: "Extract every predictive maintenance case reported in this paper, not in other papers cited within it, and all case-related facts expressible using the supplied ontology."

Results: With a successful repeat of S017 replacing its incomplete first attempt, the five-paper extraction yielded 59 case nodes against 17 human cases and about 72.7 minutes of recorded pipeline time. Terra was judged faster with no visible performance loss and was selected for subsequent experiments. This was a qualitative judgment rather than a measured equivalence in accuracy. Both models produced considerably more facts than the human baseline. The case definition still required improvement before a larger pilot.

E. E5: Adding an Explicit Case Definition

Rationale: The fifth experiment retained Terra and the same five papers, but expanded the instruction with an explicit description of the OPMAD case structure. The prompt was: "The very basic structure of a case starts with an instance of a `Predictive maintenance case` (subclass of `Designative Information Content Entity`), designating an instance of a `Predictive maintenance system module` (subclass of `Information Processing Artifact`). The items to be maintained are instances of the different subclasses in `Maintainable item` (equivalent to `Artifact`). Then, the models are instances of the type subclasses in the `Predictive maintenance model`. The models operate over a set of variables that are recovered from the operation of the item. These variables are instances of `Data variable` (subclass of `Descriptive Information Content Entity`). As the instances of variables are physical concepts (for example, the temperature), they can be reused in more than one case of the database. For each one of the different measurable magnitudes that are mentioned in the database, there exists one single instance of `Data variable`. The functioning of the module and the models used for a certain case are explained in the corresponding `Predictive maintenance article`. Extract every case reported in this article, not in other papers articles within it, and all case-related facts expressible using the supplied ontology."

Results: All five papers completed in about 75 minutes, with 55 case nodes against 17 human cases. The previous Terra prompt yielded 59 nodes. The expanded definition reduced the total by four, while substantial case-node overproduction remained. S001 remained below the human count, whereas the other four papers remained above it.

F. E6: Inspecting Rejected Entity Merges

Rationale: The sixth experiment repeated S003 with Terra and enabled detailed diagnostic logging to investigate why case nodes remained duplicated after merging. It kept the case-definition prompt verbatim: "The very basic structure of a case starts with an instance of a `Predictive maintenance case` (subclass of `Designative Information Content Entity`), designating an instance of a `Predictive maintenance system module` (subclass of `Information Processing Artifact`). The items to be maintained are instances of the different subclasses in `Maintainable item` (equivalent to `Artifact`). Then, the models are instances of the type subclasses in the `Predictive maintenance model`. The models operate over a set of variables that are recovered from the operation of the item. These variables are instances of `Data variable` (subclass of `Descriptive Information Content Entity`). As the instances of variables are physical concepts (for example, the temperature), they can be reused in more than one case of the database. For each one of the different measurable magnitudes that are mentioned in the database, there exists one single instance of `Data variable`. The functioning of the module and the models used for a certain case are explained in the corresponding `Predictive maintenance article`. Extract every case reported in this article, not in other papers articles within it, and all case-related facts expressible using the supplied ontology."

Results: The paper was processed as 13 content units and yielded 11 case nodes against one human case in about 18.5 minutes. Candidate case merges with high embedding similarity were rejected because the names did not match closely enough or because a relationship expected to have one target pointed to different identifiers. Different wording and chunk-specific identifiers for the same systems can therefore preserve multiple representations of one case, even when the extracted links follow the ontology. Chunk sizing was configurable, but the default text-joining step stayed within neighboring chunks sharing a section label, and the RDF merger's checks could not all be disabled through configuration.

G. E7: Joining Sections into Larger Chunks

Rationale: The seventh experiment repeated S003 with Terra, enabled cross-section text merging, and raised the chunk bounds

from 3,000–12,000 to 48,000–96,000 characters. It retained section filtering and detailed diagnostic logging. The modified procedure joins neighboring content across section labels while keeping separately routed bibliography material apart. The extraction prompt was unchanged: "The very basic structure of a case starts with an instance of a `Predictive maintenance case` (subclass of `Designative Information Content Entity`), designating an instance of a `Predictive maintenance system module` (subclass of `Information Processing Artifact`). The items to be maintained are instances of the different subclasses in `Maintainable item` (equivalent to `Artifact`). Then, the models are instances of the type subclasses in the `Predictive maintenance model`. The models operate over a set of variables that are recovered from the operation of the item. These variables are instances of `Data variable` (subclass of `Descriptive Information Content Entity`). As the instances of variables are physical concepts (for example, the temperature), they can be reused in more than one case of the database. For each one of the different measurable magnitudes that are mentioned in the database, there exists one single instance of `Data variable`. The functioning of the module and the models used for a certain case are explained in the corresponding `Predictive maintenance article`. Extract every case reported in this article, not in other papers articles within it, and all case-related facts expressible using the supplied ontology."

Results: The paper became one content unit of 61,342 characters instead of 13 units, and the output contained one case node, matching the one human case. Recorded pipeline time was about 4.5 minutes. This is a case-count match on one paper, rather than a full validation of its facts. Because cross-section merging and larger size bounds were applied together, this experiment does not isolate their individual effects.

REFERENCES

[1] F. Lu, J. Wu, J. Huang, and X. Qiu, "Aircraft engine degradation prognostics based on logistic regression and novel OS-ELM algorithm," *Aerospace Science and Technology*, vol. 84, pp. 661–671, 2019, doi: 10.1016/j.ast.2018.09.044.

[2] N. Vafaei, R. A. Ribeiro, and L. M. Camarinha-Matos, "Fuzzy early warning systems for condition based maintenance," *Computers & Industrial Engineering*, vol. 128, pp. 736–746, 2019, doi: 10.1016/j.cie.2018.12.056.

[3] Y. Wu, E. Breaz, F. Gao, D. Paire, and A. Miraoui, "Nonlinear performance degradation prediction of proton exchange membrane fuel cells using relevance vector machine," *IEEE Transactions on Energy Conversion*, vol. 31, no. 4, pp. 1570–1582, 2016, doi: 10.1109/TEC.2016.2582531.

[4] A. Hussain, S.-J. Lee, M.-S. Choi, and F. Brikci, "An expert system for acoustic diagnosis of power circuit breakers and on-load tap changers," *Expert Systems with Applications*, vol. 42, pp. 9426–9433, 2015, doi: 10.1016/j.eswa.2015.07.079.

[5] L. Ren, Y. Sun, H. Wang, and L. Zhang, "Prediction of bearing remaining useful life with deep convolution neural network," *IEEE Access*, vol. 6, pp. 13041–13049, 2018, doi: 10.1109/ACCESS.2018.2804930.

TABLE I
CASE-NODE COUNTS BY EXPERIMENT AND PAPER, WITH RECORDED LLM PIPELINE TIME. BLANK PAPER CELLS INDICATE PAPERS NOT TESTED IN THAT EXPERIMENT.

		S001 [1]		S003 [2]		S004 [3]		S007 [4]		S017 [5]		
Experiment	Model	Human	LLM	Human	LLM	Human	LLM	Human	LLM	Human	LLM	Time (min)
E1	Sol	6	21									NR
E2	Sol	6	4									8.0
E3	Sol			1	20	3	22	4	13	3	9	74.6
E4	Terra	6	4	1	14	3	25	4	9	3	7	72.7
E5	Terra	6	5	1	10	3	19	4	14	3	7	74.6
E6	Terra			1	11							18.5
E7	Terra			1	1							4.5

Human counts are the fixed reference extracted by the OPMAD authors, not new extractions for each experiment. LLM counts are distinct subjects explicitly typed as OPMAD predictive maintenance cases in the final merged RDF graphs. Duplicate representations of a case remain, and provenance statements are excluded. Time sums the recorded conversion, chunking, rendering, merging, validation, and serialization stages across the papers in each row, excluding initialization, nested timing counters, and failed attempts. Human extraction time was not recorded.

E1–E7 correspond to the experiment subsections. E1 counts use a rerun with saved LLM responses rather than new model calls. NR means the original runtime was not recorded. E2 includes 13 cached responses and three fresh calls. E4 uses the successful repeat of S017 in place of its incomplete first attempt.